USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Milestone 1 Verification Report

UM-CYDOS-008

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-CYDOS-008	1.0 · 2026-08-14	**PASS** — all exit criteria met on hardware	Internal

1. Abstract

Milestone 1 establishes that cyd-os can be interrupted and resume correctly: a vector table is installed, a periodic timer interrupt is dispatched, and code interrupted mid-execution continues with its register state intact.

The third property is the one that matters. Every scheduler is built on it, and a defect in the interrupt save/restore path does not announce itself — it surfaces later as corruption in unrelated code. M1 therefore measures it rather than assuming it.

2. Design decisions

2.1 Timer source — core comparator, not a peripheral

The ESP32 offers two families of timer: the TIMG peripherals, and the core's own CCOUNT/CCOMPARE comparators. **CCOMPARE1 was selected.**

	TIMG peripheral	CCOMPARE1 (selected)
Clock gating	Required	None — inside the core
Interrupt matrix routing	Required	None — fixed internal interrupt
Prescaler / config registers	Several	One comparator register
Ways to be wrong about something other than interrupts	Many	Few

For the first interrupt on a kernel with no driver model, minimising unrelated setup is worth more than the peripheral's extra features. A TIMG source is the correct long-term choice and can replace this once L3 exists.

CCOMPARE1 raises **internal interrupt 15**, which is **level 3** on this core.

2.2 Interrupt level — 3, not 1

Level 1 interrupts on Xtensa are dispatched through the general exception vector with `EXCCAUSE = 4`, requiring the handler to distinguish interrupts from genuine exceptions. Levels 2 and above have **dedicated vectors** and are entered only for that class of event.

Level 3 was therefore chosen: the handler needs no `EXCCAUSE` decoding, and the exception vectors stay free for the panic path (§2.4).

2.3 Comparator is one-shot

CCOMPARE has no auto-reload. The handler must recompute and rewrite the next deadline, and **that write is also the interrupt acknowledgement** — there is no separate acknowledge step. Forgetting it produces immediate re-entry rather than a missed tick, which is a loud failure and therefore an acceptable one.

2.4 Panic handler — added beyond the milestone

The kernel exception, user exception, and double exception vectors are populated with a handler that captures `EXCCAUSE`, `EPC1` and `PS`, prints them with the cause code decoded to a name, and halts.

Rationale: **no JTAG probe is available yet.** Without this, any unexpected fault produces a silent reset with no evidence. The handler runs on a dedicated 512-byte stack because the faulting stack may be the cause of the fault, and it halts rather than resetting so the output is not scrolled away by a boot loop.

3. Implementation

File	Contents
`kernel/vectors.S`	Vector slots, level-3 interrupt entry/exit, panic entry
`kernel/xtensa.h`	Special-register accessors with required synchronisation
`kernel/timer.c/.h`	CCOMPARE1 arming, ISR, tick and diagnostic counters
`kernel/panic.c/.h`	Fault decode and report
`kernel/linker.ld`	`.vectors` output section with architectural offsets

3.1 Vector placement

Vectors dispatch to fixed offsets from `VECBASE`, so slots are placed by absolute position in the linker script, not by declaration order. `VECBASE` must be 1024-byte aligned, which is why `.vectors` leads the image.

Each slot is 64 bytes and contains a single `j` to a handler in `.text`. A jump was used rather than an address load because `j` has ±128 KB range — sufficient for the whole kernel — and avoids needing a literal pool inside a fixed-offset slot.

3.2 Interrupt entry and exit

`a0`	return address
`a2–a11`	caller-saved
`a12–a15`	callee-saved — must travel with the task
`SAR`	shift amount
`EPC3`	interrupted PC — the switch depends on this
`EPS3`	interrupted PS
`LBEG / LEND`	zero-overhead LOOP bounds
`LCOUNT`	iterations left — a task can be suspended mid-loop

`21 words · 96 bytes with padding · 16-byte aligned`

Figure 1. *Saved-context frame. EPC3/EPS3 are what make a stack swap into an actual task switch.*

Hardware saves **nothing** on entry. It records the interrupted PC in `EPC3`, the processor state in `EPS3`, raises `PS.INTLEVEL`, and jumps. Everything else is the handler's responsibility.

The handler saves, on the interrupted context's stack:

- `a0` — return address
- `a2` – `a11` — caller-saved under call0
- `SAR` — clobbered by any shift operation

`a12` – `a15` are callee-saved under call0 and are preserved by the C handler itself, so they are deliberately not saved in assembly. Frame is 64 bytes: 12 words used, padded to keep the stack 16-byte aligned.

Return is `RFI 3`, which restores PC and PS from `EPC3` / `EPS3` atomically.

This is the entire context-save cost of the call0 decision (UM-CYDOS-003). Under the windowed ABI the same handler would additionally have to spill live register windows before touching the register file.

3.3 Synchronisation

Several special registers require a synchronising instruction before the write takes architectural effect. These are built into the accessors in `xtensa.h` rather than left to call sites, because omitting one produces a failure several instructions later:

Register	Required after write
`VECBASE`	`ISYNC` — affects instruction fetch
`PS`	`RSYNC`
`INTENABLE`, `CCOMPARE1`	`ESYNC`

4. Verification method

4.1 Pre-flight — vector placement checked before flashing

A misplaced vector produces a silent reset with no output, which is close to undiagnosable without a debugger. Placement was therefore verified in the linked ELF using `nm` **before** the image was flashed:

```
_vecbase = 0x40080000   1024-aligned: True

symbol             address      offset   expected
_vector_level3     0x400801C0   0x01C0   OK
_vector_kernel     0x40080300   0x0300   OK
_vector_user       0x40080340   0x0340   OK
_vector_double     0x400803C0   0x03C0   OK

_handler_level3 = 0x40080924   (1892 bytes from vector; j range ~128KB)
```

4.2 Register integrity under interruption

Six caller-saved registers are loaded with distinct non-zero patterns and verified immediately afterwards, in a continuous loop. Because ticks fire asynchronously, a large fraction of iterations are interrupted mid-sequence — which is exactly the case a faulty save or restore would corrupt.

Distinct patterns (`0x11111111` , `0x22222222` , …) were used so that a stray zero, or a neighbouring register's value, is unmistakable rather than plausible.

4.3 Rate measurement

The kernel does not know the CPU frequency, so the interval is expressed in cycles and the real rate is derived by counting ticks against the **host's** clock during a capture window of known duration.

5. Results

Captured 2026-08-14, target board on COM5, 10-second window.

```
  .data loaded : ok
  .bss cleared : ok
  stack top    : 0x3ffdc200
  code at      : 0x4008046c  (IRAM ok)
  vecbase      : 0x40080000  (installed)
  tick every   : 2400000 cycles
  intenable    : 0x00008000
  ps           : 0x00060720

  waiting for first tick... arrived

  tick 25  delta=2400030cy  late=0  regchecks=1556680  corrupt=0
```

#	Exit criterion	Result
1	Tick counter advances at a stable rate	**PASS** — `delta = 2,400,030` cycles against 2,400,000 requested
2	Interrupted code resumes correctly	**PASS** — 1,556,680 checks, **0** corruptions
3	System survives without reset	**PASS** — full capture window, no panic, no reboot

5.1 Interpretation

Handler overhead is 30 cycles. The measured interval exceeds the requested one by a constant 30 cycles, which is the prologue cost before `CCOUNT` is sampled. Constant rather than growing, so there is no cumulative drift.

`late = 0` — no deadline was ever missed, so the handler completes well within one tick period.

`intenable = 0x00008000` — bit 15 only, confirming CCOMPARE1 is the sole enabled source.

`ps = 0x00060720` — `INTLEVEL = 0`, so nothing is masked. The upper bits are `WOE` / `CALLINC` left by the ROM; harmless, as call0 code never consults them.

5.2 Derived CPU frequency

25 ticks occurred within the ~10 s window at 2,400,000 cycles per tick, implying a CPU clock in the region of **80 MHz** — not the 240 MHz maximum. The bootloader leaves the core at its default and nothing in the kernel raises it.

This is a **measurement of the current configuration**, not a specification figure, and it should be re-derived rather than assumed if boot configuration changes. It also explains the apparent slowness of the M0 spin-loop heartbeat.

6. What M1 does not establish

- **No task switching.** One context only; nothing is saved or restored across independent stacks.
- **Single interrupt source.** Nesting, priority interaction, and level-1 dispatch are all unexercised.
- **Panic handler untested on a real fault.** It is present and links, but no exception has been deliberately triggered to confirm the output path works under fault conditions. **Recommended before M2**, since M2 is when it will first be needed in anger.
- **Watchdogs — CORRECTED 2026-08-14.** This report originally recorded the watchdog state as "inference, not measurement", and the inference was wrong. The RTC watchdog **is** armed by the bootloader. It was later measured directly: every boot after the first reported st:0x10 (RTCWDT_RTC_RESET). M0 and M1 survived their capture windows by luck of timing, not because no watchdog was running. See \kernel/watchdog.c.
- **No CPU frequency control.** The kernel neither reads nor sets the clock.

7. Metrics

	M0	M1
`.text`	1,124 B	3,340 B
`.data`	4 B	4 B
`.bss`	4 B	544 B (panic stack)
Image	1,216 B	3,424 B

Growth of ~2.2 KB for the vector table, interrupt entry/exit, timer driver and panic handler.

8. References

- UM-CYDOS-003 §6 — why the context save is short under call0
- UM-CYDOS-004 — memory map; `.vectors` now leads the IRAM region
- UM-CYDOS-006 — M0 verification, whose assertions still pass here
- UM-CYDOS-007 §3 — M1 as originally specified